



南方科技大学

LAB 2: PyTorch

COE 201 - AI Large Models, Spring 2026

2026-05-17

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

Vectorization and Tensors	1
What is PyTorch?	6
Computational Graph and Autograd	9
Build a Neural Network with PyTorch	15
Lab Tasks	21
Other References	28

Vectorization and Tensors

Why Vectorization?



The Problem with Python Loops

```
1 # Slow: Python loop with type checking at each step
2 result = []
3 for i in range(1000000):
4     result.append(a[i] + b[i])
5
6 # Fast: Vectorized operations (C-level, contiguous memory)
7 result = a + b # 100x+ faster!
```

 Python

Why the difference?

- Python loops: CPU synchronous execution, processing elements one by one.
- Vectorized operations: Hardware-level parallelism for simultaneous execution.

Even Better on GPU



GPUs have **thousands of cores** designed for parallel computation:

	CPU	GPU
Cores	8-16	thousands
Best for	Sequential tasks	Parallel tasks
Example	Web browsing, Office apps	Matrix multiplication

Vectorized code → GPU can process thousands of elements **simultaneously**

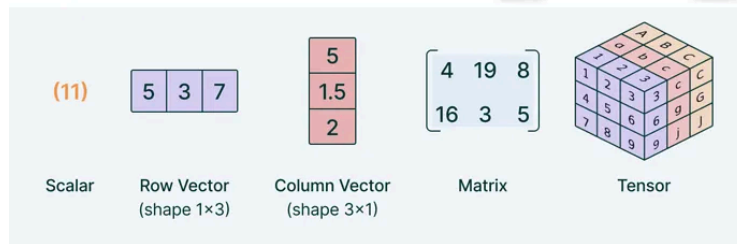
What is a Tensor?



To leverage vectorization, we need a **data structure** that stores data in contiguous blocks of memory.

Tensor dimensions:

- **Scalar** (0-D): single value
- **Vector** (1-D): list of values
- **Matrix** (2-D): table of values
- **Tensor** (N-D): N-dimensional array



Tensor = multi/any-dimensional array designed for vectorized operations.

Tensor Operations



```
1  # Creating tensors
2  x = torch.zeros(3, 3)      # All zeros
3  x = torch.ones(3, 3)       # All ones
4  x = torch.randn(3, 3)      # Random normal distribution
5  x = torch.arange(0, 10, 2)  # [0, 2, 4, 6, 8]
6
7  # Basic operations (element-wise)
8  a = torch.tensor([1., 2., 3.])
9  b = torch.tensor([4., 5., 6.])
10 print(a + b)               # tensor([5., 7., 9.])
11 print(a * b)               # tensor([4., 10., 18.])
12 print(a @ b)               # tensor(32.) - dot product
13
14 # Reshaping
15 x = torch.arange(12)
16 x = x.reshape(3, 4)        # 3 rows, 4 columns
17 x = x.view(2, 6)           # Different shape, same data
```

 Python

What is PyTorch?

PyTorch: A Deep Learning Framework

PyTorch is a tensor library with two key features:

1. **GPU Acceleration** - Run computations on NVIDIA (and others) GPUs
2. **Automatic Differentiation** - Compute gradients automatically

```
1  import torch
2
3  # Check if GPU is available
4  print(torch.cuda.is_available()) # True if GPU available
5
6  # Move tensor to GPU
7  x = torch.randn(1000, 1000)
8  x_gpu = x.to('cuda') # or x.cuda()
9
10 # Computation happens on GPU
11 y_gpu = x_gpu @ x_gpu.T
```

 Python

NumPy vs PyTorch



Both are **vectorization libraries** for fast numerical computation. PyTorch adds features for deep learning:

Feature	NumPy	PyTorch
Vectorization	✓	✓
GPU Support	✗	✓
Auto Gradient	✗	✓
Neural Network Modules	✗	✓ (nn.Module)
Optimizers	✗	✓ (torch.optim)

Computational Graph and Autograd

Backpropagation: The Core of Deep Learning

How do neural networks learn? In one iteration:

1. **Forward pass**: Input $\rightarrow$ Network $\rightarrow$ Prediction
2. **Compute loss**: Measure how far off the prediction is from the truth.
3. **Backward pass**: Compute the gradients of the loss with respect to all weights.
4. **Update weights (Optimization)**: Adjust weights in the opposite direction of the gradient to minimize the loss.

Backpropagation (step 3) is the algorithm that makes deep learning possible.

- It efficiently computes gradients for **millions** of parameters.
- Without it, training deep neural networks would be computationally impossible.

Chain Rule: The Math Behind Backprop



To compute gradients through a chain of operations, we use the **chain rule**:

$$\frac{dy}{dx} = \frac{dy}{dz} \cdot \frac{dz}{dx}$$

Example: Given $y = z^2$, $z = x + 2$. If $x = 3$, calculate $\frac{dy}{dx}$.

Forward pass:

- Given $x = 3$
- Then $z = x + 2 = 5$
- Finally $y = z^2 = 25$

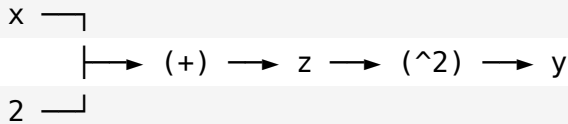
Backward pass:

- $\frac{dy}{dz} = 2z = 10$
- $\frac{dz}{dx} = 1$
- $\frac{dy}{dx} = 10 \times 1 = 10$

How can we track these operations and compute gradients automatically?

A **computational graph** is a Directed Acyclic Graph (**DAG**) that tracks a sequence of operations.

Same example: input x , then $z = x + 2$, and output $y = z^2$.



So now we know how the data flows:

- **Forward pass:** Build the graph, input $x = 3$, and compute $z = 5$, $y = 25$.
- **Backward pass:** Traverse the graph in reverse, applying the chain rule at each node.

This same principle scales to neural networks with **millions** of operations!

Same example in PyTorch: $y = z^2$, $z = x + 2$, $x = 3$

```
1  import torch
2
3  # Create tensor with gradient tracking
4  x = torch.tensor(3.0, requires_grad=True)
5
6  # Forward pass: PyTorch builds the computational graph automatically
7  z = x + 2      # z = 5
8  y = z ** 2     # y = 25
9
10 # Backward pass: Compute gradients
11 y.backward()
12
13 # Gradient is stored in .grad attribute
14 print(x.grad)  # dy/dx = 10.0 ✓ (same as our manual calculation!)
```

 Python

Detach and No Grad



```
1  # .detach() - Remove tensor from computation graph
2  y = w * x
3  y_detached = y.detach() # No longer tracks gradients
4
5  # torch.no_grad() - Disable gradient computation (inference)
6  with torch.no_grad():
7      y = model(x) # Faster, less memory
8
9  # Common use: updating weights manually
10 w.data -= learning_rate * w.grad # Don't track this update!
11 w.grad.zero_() # Reset gradients for next iteration
```

 Python

Build a Neural Network with PyTorch

Using nn.Module

```
1  import torch
2  import torch.nn as nn
3
4  class SimpleNet(nn.Module):
5      def __init__(self, input_size, hidden_size, output_size):
6          super().__init__()
7          self.layer1 = nn.Linear(input_size, hidden_size)
8          self.relu = nn.ReLU()
9          self.layer2 = nn.Linear(hidden_size, output_size)
10
11     def forward(self, x):
12         x = self.layer1(x)
13         x = self.relu(x)
14         x = self.layer2(x)
15         return x
16 # Create model
17 model = SimpleNet(784, 128, 10)
```

 Python

Using nn.Sequential (Simpler)



```
1  # For simple feed-forward networks (e.g., MNIST digit classification)
2  model = nn.Sequential(
3      nn.Flatten(),
4      nn.Linear(784, 128),
5      nn.ReLU(),
6      nn.Linear(128, 64),
7      nn.ReLU(),
8      nn.Linear(64, 10)
9  )
10
11 # Forward pass
12 import torch
13 x = torch.randn(32, 1, 28, 28) # Batch of 32 images, 1 channel, 28x28
14 output = model(x)
15 print(output.shape)           # torch.Size([32, 10])
```

 Python

Optimization algorithms determine **how** to update the model weights using their computed gradients to minimize the loss.

- **Gradient Descent (GD)**: Calculates the gradient using the **entire training dataset**.
 - **Pros**: Provides a stable and accurate gradient direction.
 - **Cons**: Extremely slow; computes over the full dataset just to make a single update.
- **Stochastic Gradient Descent (SGD)**:
 - Originally refers to updating weights using just **a single sample** at a time ($\eta \nabla L(x_i, y_i)$).
 - **Pros**: Enables very fast updates.
 - **Cons**: High variance; the gradient direction is noisy and fluctuates significantly.

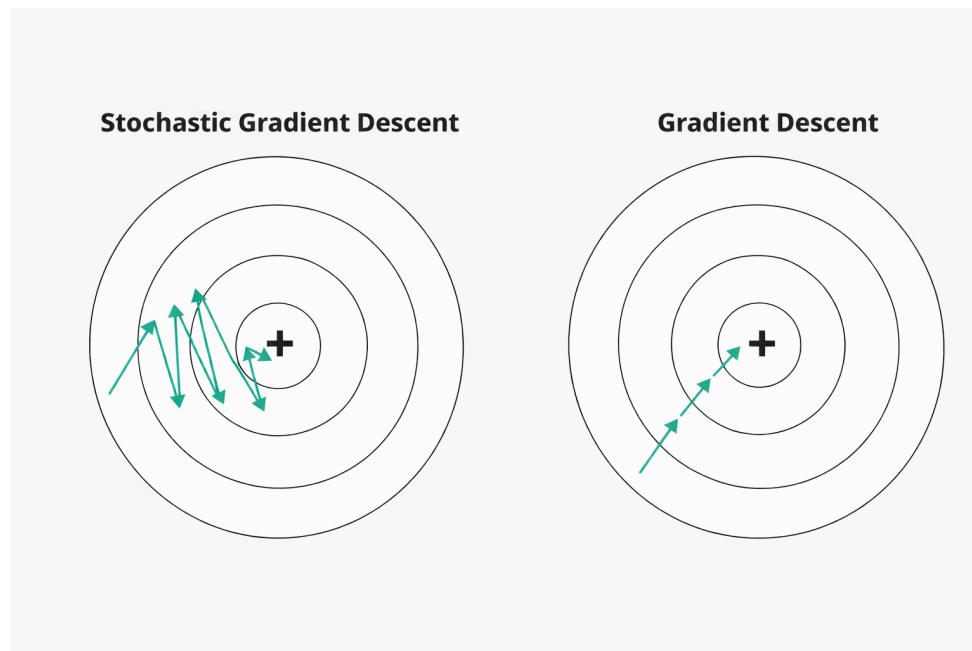
SGD vs. GD: Tradeoff Visualization



南方科技大学

Tradeoff of optimizer in a very simple example:

- SGD: Fast but not so accurate on each step
- GD: Slow but accurate on each step



In modern Deep Learning, “SGD” almost always refers to **Mini-Batch SGD**.

- Divides the dataset into small batches (e.g., 32, 64, or 128 samples).
- Combines the **speed/memory efficiency** of SGD with the **stability** of full GD.
- In PyTorch, a `DataLoader` yields these mini-batches automatically!

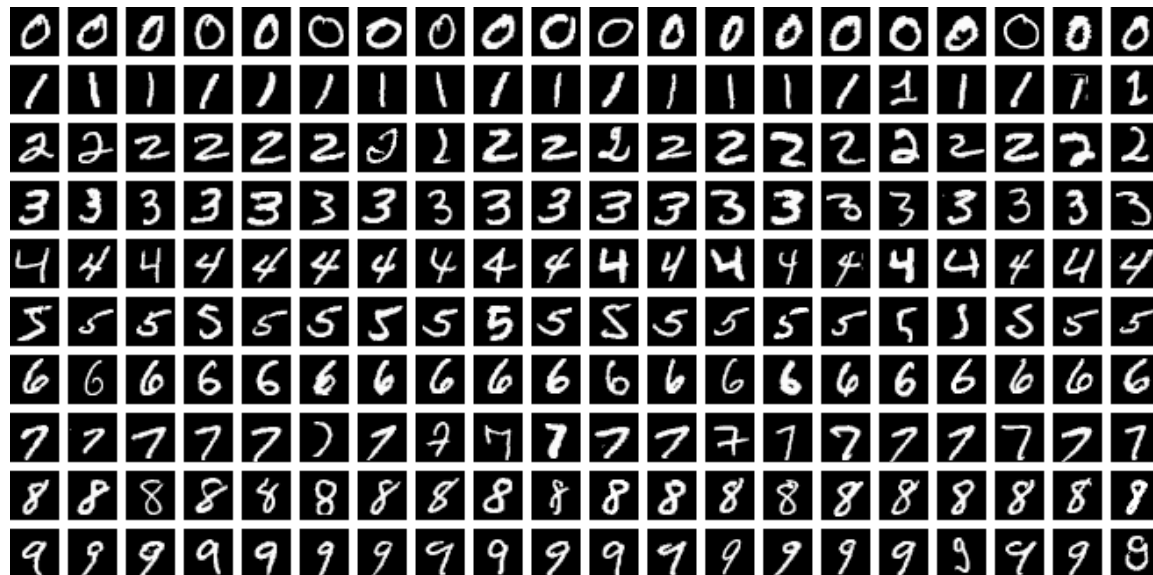
A simplest train loop with PyTorch

 Python

```
1 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
2 # Training loop with Mini-Batch SGD
3 for images, labels in train_loader: # train_loader yields batches of 32
4     outputs = model(images)
5     loss = criterion(outputs, labels) # Loss over the mini-batch
6     optimizer.zero_grad() # 1. Clear previous gradients
7     loss.backward() # 2. Compute gradients for this batch
8     optimizer.step() # 3. Update weights:  $w = w - lr * grad$ 
```

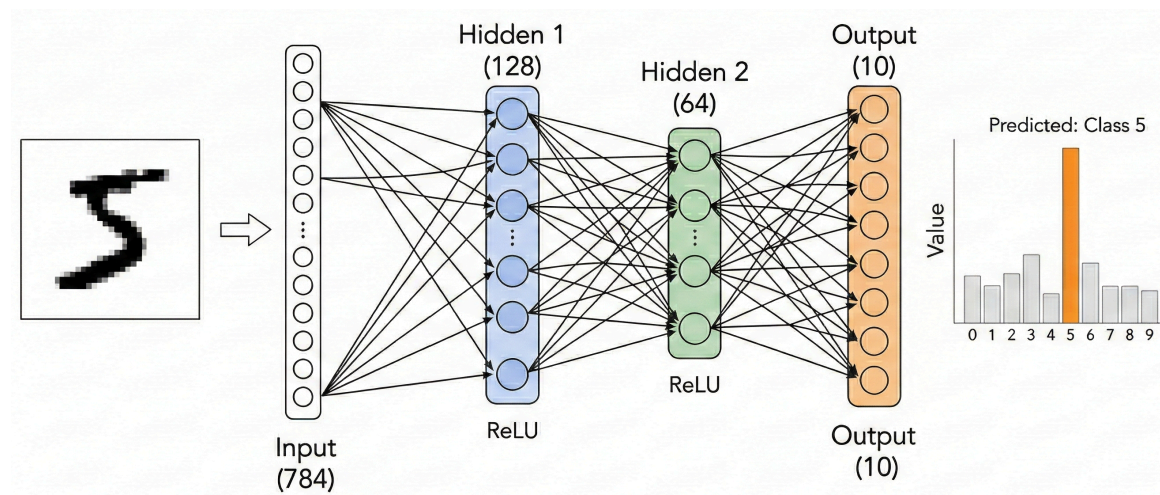
Lab Tasks

- **MNIST Dataset:** A classic dataset of handwritten digits (0-9).
 - 60,000 training images, 10,000 testing images.
 - Each image is a 28×28 grayscale pixel matrix.
 - Task: Train a Neural Network to classify the image into one of the 10 digits (0-9).



- **Multi-Layer Perceptron (MLP):**

- ▶ **Input Layer:** Flattens the 28×28 image into a 1D tensor of length 784.
- ▶ **Hidden Layers:** Fully connected linear layers followed by activation functions.
- ▶ **Output Layer:** Final linear layer outputting 10 values (logits), representing the scores for each digit class.



The sections you need to implement are marked with **### TODO** in the code.

Task 1: Manual Gradient Descent (`lab2_manual_gd.py`) [40 points]

- Target: $y_p = w * x + b$; Loss: $\text{MSE} = \frac{1}{N} \sum (y_p - y_t)^2$
- Update logic: $w = w - \eta \cdot \nabla w$, $b = b - \eta \cdot \nabla b$
- Manual steps: `.backward()`, `.grad`, `.zero_()`, and weight update in `torch.no_grad()`.

Task 2: Build a 3-Layer MLP for MNIST (`lab2_mnist_mlp.py`) [60 points]

- Build a feed-forward model using `nn.Sequential`.
- Implement a complete training loop with a PyTorch optimizer.

Optional Task: Swish Activation



南方科技大学

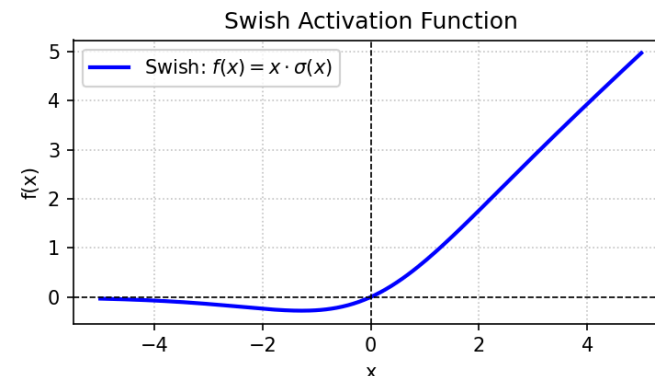
Task 3 (Optional): Custom Autograd Function (`lab2_opt1_custom_autograd.py`) [Bonus]

- Manually implement the Swish Activation function and its derivative.

Swish: $f(x) = x \cdot \text{sigmoid}(x) = \frac{x}{1+e^{-x}}$

Hints: Derivative of sigmoid:

$$\text{sigmoid}'(x) = \text{sigmoid}(x)(1 - \text{sigmoid}(x))$$



Thinking: Why might we need to implement the backward pass manually if PyTorch can do it automatically with autograd?

Optional Task: Polynomial Regression



南方科技大学

Task 4 (Optional): Polynomial Regression & Optimizers (`lab2_opt2_poly_regression.py`) [Bonus]

- Construct a non-linear prediction function ($y = a + bx + cx^2 + dx^3$) using `requires_grad=True`.
- Implement training loops for full GD, Mini-Batch SGD, and Stochastic GD.
- Adjust learning rates to achieve stable training.

Quick reference for common Deep Learning terminology:

- **Batch Size**: The number of training samples processed in one forward/backward pass (refer to the Mini-Batch SGD & PyTorch Optimizer slide).
- **Iteration (Step)**: A single update of the model's parameters (1 iteration = processing 1 batch).
- **Epoch**: One complete pass through the **entire** training dataset.
 - If you have 60,000 images and a batch size of 60, it takes 1,000 **Iterations** to complete 1 **Epoch**.
- **Learning Rate (η , LR)**: A tiny scalar (e.g., 0.01) that determines how large the steps are when updating the weights.

Other References

Some useful resources:

- Calculus Basics (Gradient, Derivative, Chain Rule): [link](#)
- Backpropagation and Autograd: [link](#)
- Gradient Descent: [link](#)

Thanks for Listening!